Mavlyudabegim Mekhrilloeva

M.Sc. Computer Science, Saarland University

Saarbrücken, Germany · +49 157 50730857 · mekhrullaeva@gmail.com
LinkedIn · GitHub · Website

Profile

M.Sc. Computer Science student (Machine Learning focus) at Saarland University and DAAD scholar with 2 years of professional experience as a software engineer, building reporting tools for insurance companies and financial institutions. Skilled at translating business requirements into practical technical solutions and explaining technical concepts to non-technical stakeholders; passionate about applying AI and automation to real business problems.

Skills

AI & Data	Machine Learning, Neural Networks, NLP / Machine Translation, Reinforcement Learning, computer vision (segmentation), data analysis, data quality, PyTorch, NumPy, pandas, scikit-learn
LLMs & RAG	Retrieval-augmented generation, hybrid search (embeddings + BM25), prompt design, LLM evaluation and error analysis
Programming & Data	Python, SQL, PostgreSQL, TypeScript/JavaScript, Node.js
Frameworks & Tools	Django (ORM), NestJS, Angular (NgRx), React, Git, Docker, Kubernetes, AWS, Supabase, SonarQube, Jira
Quality & Testing	Unit and end-to-end testing (Jest), debugging
Office	MS Office (Excel, Word, PowerPoint, Outlook)
Soft Skills	Stakeholder communication, requirements analysis, analytical problem-solving, teamwork, independent work

Professional Experience

Software Engineer (Python / Angular) — Exadel 06/2022 – 04/2024 · Tashkent, Uzbekistan

- Developed a web-based reporting platform for insurance companies and financial institutions to build regulatory and financial reports from ready-made building blocks.
- Translated client business requirements into technical solutions by tailoring core form designs into custom financial reports with spreadsheet-like formulas and calculation features.
- Automated report generation with Python/Django services on a PostgreSQL database, producing Excel and PDF documents, reducing manual work for business users.
- Built data-heavy Angular features with NgRx state management (virtual scroller for large datasets, financial report tables), improving usability by 65%.
- Improved data quality and reliability: used SonarQube for automated code analysis and fixed validation, authorization and API errors on server and UI side; wrote unit and end-to-end tests with Jest, decreasing bugs in production by 70%.
- Collaborated with stakeholders in an international, cross-functional team of 15 people.

Software Engineering Intern (Node.js / Angular) — EPAM Systems Tashkent, Uzbekistan

- Worked on a litter-picking web application for the city of Tashkent: residents report places with garbage, organise clean-up events and find volunteers nearby.
- Built backend features with NestJS and Node.js (REST APIs for reports, events and volunteer sign-up) and frontend features in Angular.

Education

M.Sc. Computer Science (Informatik) — Saarland University, Saarbrücken 10/2024 – expected 01/2027

- All coursework completed; thesis in progress: *Delegated Classification — contract design for ML as a Stackelberg game*
- Relevant courses: Machine Learning, Neural Networks, Machine Translation (NLP), Optimization Algorithms in ML, Cybersecurity in ML
- DAAD Scholarship holder (All Disciplines)

B.Sc. Computer Engineering — Tashkent University of Information Technologies 2018 – 2022

- Grade: 1.1 (German grading scale) · Thesis: *Algorithms for preprocessing EMG biosignals*
- Relevant courses: Data Structures and Algorithms, Machine Learning, Artificial Intelligence, Numerical Methods

PROJECTS

Legal Document Navigator — LLM-based RAG for contracts Python, PyMuPDF, Supabase, Streamlit · GitHub

- Built an end-to-end RAG application: upload a contract (PDF), ask questions in plain language and get short answers that cite the exact clause, with the supporting sentences highlighted in the PDF viewer.
- Implemented layout-aware PDF parsing (headings from font size, header/footer removal) and clause-based chunking with section heading paths; hybrid retrieval with bge-small embeddings and BM25 merged by Reciprocal Rank Fusion; storage in Supabase (Postgres + pgvector) with a local Chroma fallback.
- Grounded generation with verified citations: references to clauses the model never saw are removed, and the model answers "not found" instead of guessing; runs gpt-oss-20b locally (Ollama) or gpt-oss-120b (Groq).
- Evaluated on ContractNLI (61 NDAs labelled by lawyers; fixed 300-question sample, paired McNemar tests): **96.1% Hit@5** and **94.0% evidence recall** in retrieval; **98.4%** of yes/no answers cite the clause that proves them; **95.2%** citation recall — without any task-specific training.
- Error analysis showed that reading the clause, not search, was the bottleneck; tested and rejected six alternatives (larger embeddings, 500-token chunks, whole-document context, few-shot prompting, more reasoning, follow-up search) with significance tests.

Scribble-Supervised Image Segmentation Python, PyTorch, scikit-learn, OpenCV · GitHub

- Binary foreground/background segmentation of 228 PASCAL VOC images using only a few rough user scribbles as labels instead of dense pixel masks.
- Propagated scribble labels with SLIC superpixels (~2,500 per image) and graph-based label spreading (RBF kernel); added distance-to-scribble input channels and a per-pixel confidence weight map.
- Trained a U-Net with an ImageNet-pretrained ResNet-34 encoder on 5-channel input (RGB + 2 distance maps) with a hybrid Focal + soft Dice loss, Adam and early stopping on validation mIoU (80/20 split); refined masks with bilateral filtering and morphological closing/opening.
- Result on the 20% held-out split: **88% mean IoU** (**95% background IoU**, 82% foreground IoU).

Tic-Tac-Toe with Q-Learning Python, NumPy, pytest, JavaScript · Live demo · GitHub

- Agent that learns perfect Tic-Tac-Toe from scratch with tabular **Q-learning** and **self-play**, receiving only the final reward (+1 win, 0 draw); one Q-table plays both X and O by viewing every board from the side of the player to move.
- Implemented the two-player (negamax) Q-learning target ($-\gamma \max Q$ of the opponent's position, $\gamma = 0.9$), backward updates from the last move to the first, ε-greedy exploration ($1.0 \to 0.05$) and a decaying learning rate ($0.5 \to 0.05$).
- Used the **8 board symmetries** (rotations and reflections) to map equivalent positions to one Q-table row: perfect play after **4,000 instead of 24,000 games (6× faster)** with a **7× smaller table** (627 vs 4,520 states), averaged over 3 seeds.
- Evaluated against fixed examiners instead of self-play scores, using an exact minimax solver for grading only: **100% optimal moves in all 4,520 positions**, **0 losses** in 2,000 games against a perfect minimax player, and 96.9% (as X) / 74.5% (as O) wins against a random player, with no losses.
- Built a Python package (train / evaluate / play / export, ablation flags, unit and end-to-end tests with pytest, fixed seeds) and a browser app to play against the trained agent with a Q-value heat map of every move.

Delegated Classification (Master's thesis) Game theory, Machine Learning

- Modelling incentive contracts between a principal and an ML agent as a Stackelberg game; developing robust contracts under delegation.

LANGUAGES & AWARDS

Languages: English (C1, fluent) · German (B1, attending language course) · Russian (C1) · Uzbek (native)
Awards: DAAD Scholarship — All Disciplines (2024)